Tugas Besar 1

IF3170 - Inteligensi Artifisial

Implementasi Algoritma Local Search



Anggota Kelompok

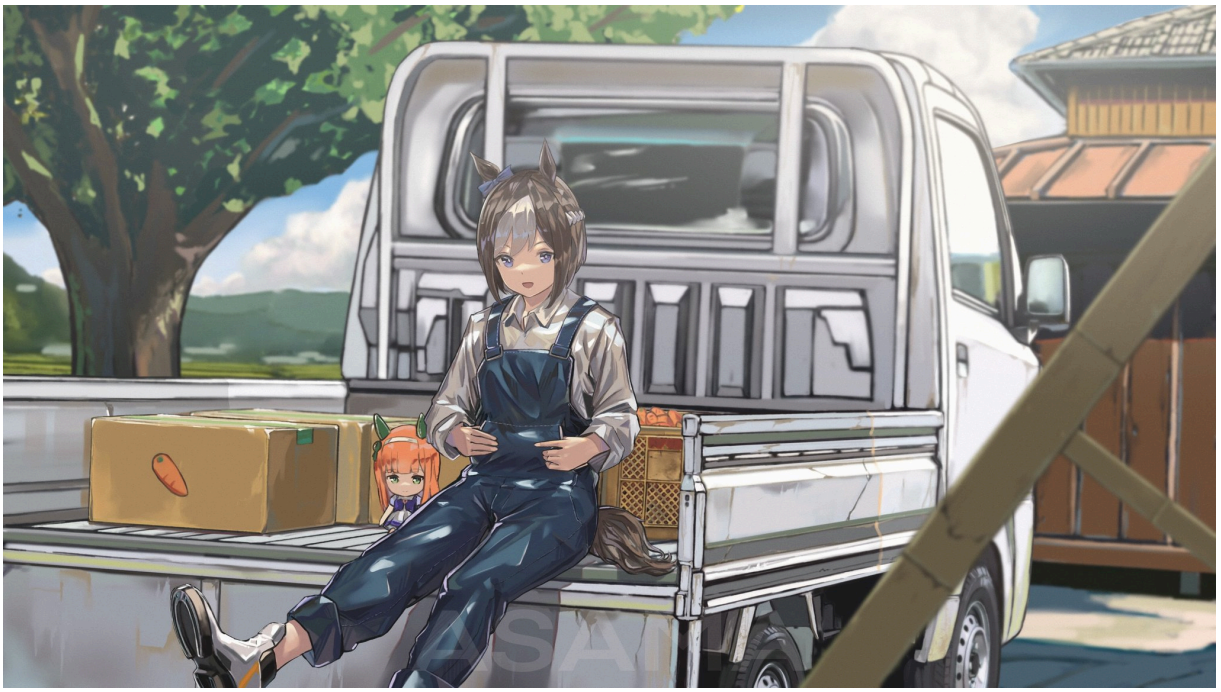
Jonathan Alveraldo Bangun - 13524120
Ramadhian Nabil Firdaus Gumay - 13524126
Jonathan Harijadi - 13524144

DAFTAR ISI

DAFTAR ISI **2**
I. Deskripsi Persoalan **3**
1.1. Latar Belakang 3
1.2. Spesifikasi 3
Daftar Variabel Permasalahan 3
Ketentuan Representasi 4
Objective Function & Constraints 5
II. Pembahasan **6**
2.1. Implementasi Algoritma 6
2.1.1. Algoritma Hill-Climbing 6
2.1.2. Algoritma Simulated Annealing 9
2.1.3. Genetic Algorithm 13
2.1.4. Komponen Umum: State, Fungsi Objektif, Constraint, dan Neighbor 22
2.1.4.1. Representasi State 22
2.1.4.2. Fungsi Objektif 25
2.1.4.3. Constraint 25
2.1.4.4. Neighbor 29
2.1.4.5. Heuristik Kandidat Posisi 30
2.2. Hasil Eksperimen 33
2.2.1. Algoritma Hill-Climbing 33
2.2.2. Algoritma Simulated Annealing 37
2.2.3. Genetic Algorithm 38
2.3. Analisis Eksperimen 40
2.3.1. Kedekatan dengan Global Optimum 41
2.3.2. Perbandingan Hasil Antar Algoritma 41
2.3.3. Perbandingan Durasi 42
2.3.4. Konsistensi Hasil 42
2.3.5. Pengaruh Iterasi dan Populasi pada Genetic Algorithm 42
III. Kesimpulan dan Saran **43**
IV. Pembagian Tugas **44**
V. Referensi **45**
VI. Lampiran Form Penggunaan Generative AI **46**

I. Deskripsi Persoalan

1.1. Latar Belakang



Gambar 1. Special Week, driver truk handal (Sumber: 真夏の银堡垒🐴C108土曜日 東地区 ホ-10a on X)

Keluarga **Mejiro** sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, **Mejiro McQueen** menawarkan posisi driver truk ujicoba kepada **Special Week**, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama **Mejiro Express** dengan slogan "Lebih kilat dari Umamusume".

Karena **Special Week** sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.

1.2. Spesifikasi

Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan

Package	id	"RusdiPackage"	*Identifier*
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu *width*, *length*, dan *height*
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu *z*.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height*
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar **(seluruh bagian paket di luar)**
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan *key-value* seperti JSON.
- Inisialisasi (***initial state***) random.
- Langkah neighbor (***move***) yang diperbolehkan tiap iterasi:
    1. Menukar dua paket, baik di dalam maupun di luar truk
    2. Memindahkan sebuah paket ke suatu koordinat yang berbeda
    3. Merotasi paket 90° terhadap sumbu *x*, *y*, atau *z*
- Representasi solusi (***final state***) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:

    1.a. Bagaimana keadaan semua paket yang di dalam maupun yang di luar

    1.b. Bagaimana keadaan peti kemas pada truk

1.c. *Value* pada *state* tersebut

Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- **Objective function**
  Permasalahan ini bertujuan untuk memaksimalkan *value* peti kemas pada truk.
- **Kardus tidak bisa melayang dan menembus dinding peti kemas**
  Melayang disini didefinisikan dengan misalkan paket i setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(x_i, y_i, z_i)$$

  dan dimensi

  $$(w_i, d_i, h_i)$$

  sehingga alas paket i adalah

  $$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i,\ y_i \leq y < y_i + d_i\})$$

  Sebuah titik di alas memiliki *support* apabila:

  1. $Z_{i=0}$, artinya di alas truk, atau
  2. Terdapat paket lain j yang mempunyai bagian atas tepat di bawah titik tersebut

  Maka melayang adalah kondisi tidak ada satupun titik alas yang memiliki support.
- **Barang pecah-belah tidak boleh di bawah paket lain**

  Barang pecah-belah tidak boleh menjadi *support*
- **Truk tidak boleh mengangkut melebihi kapasitasnya**

II. Pembahasan

2.1. Implementasi Algoritma

2.1.1. Algoritma Hill-Climbing

A. **_best_neighbor(current, packages, truck, rng):** Mengevaluasi seluruh tetangga yang dihasilkan oleh all_neighbors(). Mengembalikan tetangga dengan nilai objektif tertinggi (jika ada beberapa tetangga terbaik dengan skor sama, dipilih secara acak).

B. **_climb(packages, truck, initial, seed, max_sideways):** Fungsi inti pendakian yang digunakan oleh *Steepest Ascent* dan *Sideways Move*. Pada setiap iterasi, algoritma mencari _best_neighbor. Jika nilai tetangga terbaik lebih kecil dari state saat ini, algoritma berhenti. Jika sama (*flat*), pergerakan tetap diterima selama jumlah langkah mendatar belum mencapai max_sideways.

C. **steepest_ascent(packages, truck, initial, seed):** Memanggil _climb dengan max_sideways=0 (hanya menerima tetangga yang memiliki skor objektif lebih tinggi secara ketat).

D. **sideways_move(packages, truck, max_sideways=100, initial, seed):** Memanggil _climb dengan batas langkah mendatar maksimum max_sideways (default 100) untuk melewati *shoulder/plateau*.

E. **stochastic(packages, truck, max_iter=1000, initial, seed):** Selama max_iter iterasi, algoritma mengambil satu tetangga acak menggunakan random_neighbor(). Tetangga tersebut langsung diterima jika nilai objektifnya lebih besar dari state saat ini.

F. **random_restart(packages, truck, max_restart=10, seed):** Menjalankan steepest_ascent berulang kali dari state acak baru hingga max_restart kali (atau berhenti lebih awal jika nilai objektif mencapai total seluruh value paket), lalu menyimpan state terbaik dari seluruh percobaan.

Source Codenya :

```
import random
import time
from core.models import random_state
from core.neighbors import all_neighbors, random_neighbor
from core.objective import objective

# tetangga terbaik, kalau nilai sama dipilih acak
```

```
def _best_neighbor(current, packages, truck, rng):
    best_value, best = None, []
    for neighbor in all_neighbors(current, packages,
truck):
        value = objective(neighbor, packages, truck)
        if best_value is None or value > best_value:
            best_value, best = value, [neighbor]
        elif value == best_value:
            best.append(neighbor)
    return (rng.choice(best), best_value) if best else
(None, None)

# hill climbing dengan batas total sideways
def _climb(packages, truck, initial, seed, max_sideways):
    rng = random.Random(seed)
    start = time.perf_counter()
    current = initial.copy() if initial else
random_state(packages, truck, rng)
    initial_state = current.copy()
    current_value = objective(current, packages, truck)
    history = [current_value]
    sideways = 0
    while True:
        best, best_value = _best_neighbor(current,
packages, truck, rng)
        if best is None or best_value < current_value:
            break
        if best_value == current_value:
            if sideways >= max_sideways:
                break
            sideways += 1
        current, current_value = best, best_value
        history.append(current_value)
    return {
        "initial_state": initial_state,
        "final_state": current,
        "final_objective": current_value,
        "history": history,
        "iterations": len(history) - 1,
        "sideways_moves": sideways,
        "duration": time.perf_counter() - start,
    }

# steepest ascent hill climbing
def steepest_ascent(packages, truck, initial=None,
seed=None):
```

```
    return _climb(packages, truck, initial, seed,
max_sideways=0)

# hill climbing with sideways move
def sideways_move(packages, truck, max_sideways=100,
initial=None, seed=None):
    return _climb(packages, truck, initial, seed,
max_sideways)

# stochastic hill climbing
def stochastic(packages, truck, max_iter=1000,
initial=None, seed=None):
    rng = random.Random(seed)
    start = time.perf_counter()
    current = initial.copy() if initial else
random_state(packages, truck, rng)
    initial_state = current.copy()
    current_value = objective(current, packages, truck)
    history = [current_value]
    for _ in range(max_iter):
        neighbor = random_neighbor(current, packages,
truck, rng)
        value = objective(neighbor, packages, truck)
        if value > current_value:
            current, current_value = neighbor, value
        history.append(current_value)
    return {
        "initial_state": initial_state,
        "final_state": current,
        "final_objective": current_value,
        "history": history,
        "iterations": max_iter,
        "duration": time.perf_counter() - start,
    }

# random restart, steepest ascent diulang dari state acak
def random_restart(packages, truck, max_restart=10,
seed=None):
    rng = random.Random(seed)
    start = time.perf_counter()
    goal = sum(p.value for p in packages)
    best, history, iterations_per_restart = None, [], []
    for restart in range(max_restart + 1):
        result = steepest_ascent(packages, truck,
seed=rng.random())
        history.extend(result["history"])
```

```
iterations_per_restart.append(result["iterations"])
        if best is None or result["final_objective"] >
best["final_objective"]:
            best = result
        if best["final_objective"] == goal:
            break
    return {
        "initial_state": best["initial_state"],
        "final_state": best["final_state"],
        "final_objective": best["final_objective"],
        "history": history,
        "restarts": restart,
        "iterations_per_restart": iterations_per_restart,
        "iterations": sum(iterations_per_restart),
        "duration": time.perf_counter() - start,
    }
```

2.1.2. Algoritma Simulated Annealing

A. **Kelas SAResult (@dataclass):** Menyimpan metrik hasil eksekusi mulai dari initial_state, final_state, best_state, skor objektif, jumlah iterasi, durasi eksekusi, frekuensi *stuck* (stuck_count), hingga histori nilai objektif, temperatur, dan probabilitas penerimaan $e^{\Delta E/T}$.

B. **Kelas SimulatedAnnealing:**

- Memiliki parameter konfigurasi default initial_temp = 1000.0, cooling_rate = 0.995, min_temp = 1e-3, max_iterations = 5000, schedule_type = "geometric", dan stuck_window = 50.
- **_calculate_temperature(iteration, current_temp):** Menurunkan suhu T sesuai jadwal pendinginan (*geometric*: $T_{t+1} = T_t \times \alpha$; *linear*; atau *logarithmic*).
- **run(initial_state):** Pada setiap iterasi (selama iterasi $< max_iterations$ dan $T > T_{\min}$), algoritma membangkitkan satu tetangga acak dan menghitung $\Delta E = score_{candidate} - score_{current}$. Jika $\Delta E > 0$, kandidat langsung diterima (acceptance_prob = 1.0). Jika $\Delta E \leq 0$, kandidat diterima dengan probabilitas Boltzmann $P = e^{\Delta E/T}$. Indikator *stuck* (stuck_count) bertambah setiap kali nilai best_score tidak mengalami peningkatan selama stuck_window (50) iterasi berturut-turut.

Source Codenya :

```
import math
import random
import time
from copy import deepcopy
from dataclasses import dataclass, field
from typing import Any, Callable, List, Optional
from core.models import random_state
from core.neighbors import random_neighbor
from core.objective import objective

@dataclass
class SAResult:
    initial_state: Any
    final_state: Any
    best_state: Any
    initial_score: float
    final_score: float
    best_score: float
    iterations: int
    duration_sec: float
    stuck_count: int
    stuck_events_iterations: List[int] =
field(default_factory=list)
    objective_history: List[float] =
field(default_factory=list)
    best_objective_history: List[float] =
field(default_factory=list)
    acceptance_prob_history: List[float] =
field(default_factory=list)
    temperature_history: List[float] =
field(default_factory=list)

class SimulatedAnnealing:
    def __init__(
        self,
        objective_fn: Callable[[Any], float],
        neighbor_fn: Callable[[Any], Any],
        initial_temp: float = 1000.0,
        cooling_rate: float = 0.995,
        min_temp: float = 1e-3,
        max_iterations: int = 5000,
        schedule_type: str = "geometric",
        stuck_window: int = 50,
        seed: Optional[int] = None,
```

```
    ):
        self.objective_fn = objective_fn
        self.neighbor_fn = neighbor_fn
        self.initial_temp = initial_temp
        self.cooling_rate = cooling_rate
        self.min_temp = min_temp
        self.max_iterations = max_iterations
        self.schedule_type = schedule_type
        self.stuck_window = stuck_window
        self.seed = seed

    def _calculate_temperature(self, iteration: int,
current_temp: float) -> float:
        if self.schedule_type == "geometric":
            return current_temp * self.cooling_rate
        elif self.schedule_type == "linear":
            decrement = (self.initial_temp -
self.min_temp) / max(1, self.max_iterations)
            return max(self.min_temp, self.initial_temp -
iteration * decrement)
        elif self.schedule_type == "logarithmic":
            return self.initial_temp / math.log(iteration
+ 2)
        else:
            raise ValueError(f"Cooling schedule tidak
dikenali: {self.schedule_type}")

    def run(self, initial_state: Any) -> SAResult:
        if self.seed is not None:
            random.seed(self.seed)
        start_time = time.perf_counter()
        current_state = deepcopy(initial_state)
        current_score = self.objective_fn(current_state)
        best_state = deepcopy(current_state)
        best_score = current_score
        initial_state_copy = deepcopy(current_state)
        initial_score = current_score
        temp = self.initial_temp

        objective_history: List[float] = [current_score]
        best_objective_history: List[float] = [best_score]
        acceptance_prob_history: List[float] = [1.0]
        temperature_history: List[float] = [temp]

        stuck_count = 0
        stuck_events_iterations: List[int] = []
```

```
        no_improvement_streak = 0
        iteration = 0

        while iteration < self.max_iterations and temp >
self.min_temp:
            iteration += 1
            candidate_state =
self.neighbor_fn(current_state)
            candidate_score =
self.objective_fn(candidate_state)
            delta_e = candidate_score - current_score

            if delta_e > 0:
                acceptance_prob = 1.0
                accepted = True
            else:
                acceptance_prob = math.exp(delta_e / temp)
if temp > 0 else 0.0
                accepted = random.random() <
acceptance_prob

            if accepted:
                current_state = candidate_state
                current_score = candidate_score

            if current_score > best_score:
                best_score = current_score
                best_state = deepcopy(current_state)
                no_improvement_streak = 0
            else:
                no_improvement_streak += 1
                if no_improvement_streak >=
self.stuck_window:
                    stuck_count += 1

stuck_events_iterations.append(iteration)
                    no_improvement_streak = 0

            objective_history.append(current_score)
            best_objective_history.append(best_score)

acceptance_prob_history.append(acceptance_prob)
            temperature_history.append(temp)
            temp = self._calculate_temperature(iteration,
temp)
```

```
        duration_sec = time.perf_counter() - start_time
        return SAResult(
            initial_state=initial_state_copy,
            final_state=current_state,
            best_state=best_state,
            initial_score=initial_score,
            final_score=current_score,
            best_score=best_score,
            iterations=iteration,
            duration_sec=duration_sec,
            stuck_count=stuck_count,

stuck_events_iterations=stuck_events_iterations,
            objective_history=objective_history,
            best_objective_history=best_objective_history,

acceptance_prob_history=acceptance_prob_history,
            temperature_history=temperature_history,
        )

def run_sa(packages, truck, seed=None, **params) ->
SAResult:
    rng = random.Random(seed)
    sa = SimulatedAnnealing(
        objective_fn=lambda s: objective(s, packages,
truck),
        neighbor_fn=lambda s: random_neighbor(s, packages,
truck, rng),
        seed=seed,
        **params,
    )
    return sa.run(random_state(packages, truck, rng))
```

2.1.3. Genetic Algorithm

A. Gene & Individual (models.py): Setiap kromosom (Individual) memiliki daftar Gene sepanjang jumlah paket. Tiap Gene menyimpan status inside, koordinat (x, y, z), dan orientation.

B. Kelas GeneticAlgorithm (geneticAlgorithm.py):

- Parameter default: crossover_rate = 0.85, mutation_rate = 0.12, elite_count = 2, dan tournament_size = 3.
- **repair(individual):** Mekanisme perbaikan kromosom yang mengurutkan kandidat paket berdasarkan ketinggian z terendah dan

nilai value terbesar (key=lambda i: (z, -value, i)). Paket yang keluar batas truk, melebihi kapasitas berat, bertabrakan (*overlap*), tidak ditopang secara penuh (*full base cell support*), atau berada di atas paket fragile akan dikeluarkan (inside = False). Dengan demikian setiap individu hasil repair dijamin valid dan nilai fitness sama dengan loaded_value.

- **_select(population):** Menggunakan *Tournament Selection* dengan memilih tournament_size (3) individu secara acak dan mengambil individu dengan *fitness* tertinggi.
- **_crossover(a, b):** Menggunakan *Uniform Crossover* dengan peluang crossover_rate (0.85), di mana setiap gen ditukar antar-induk dengan peluang 0.5, lalu diperbaiki dengan repair().
- **_mutate(individual):** Setiap gen bermutasi dengan peluang mutation_rate (0.12) melalui salah satu dari tiga aksi acak: toggle (memasukkan/mengeluarkan paket), move (memindahkan ke koordinat acak), atau rotate (mengubah orientasi), kemudian diperbaiki dengan repair().
- **run():** Mengelola siklus evolusi mulai dari inisialisasi populasi awal, pencatatan histori *fitness* maksimum dan rata-rata populasi, penerapan *Elitism* (elite_count = 2 individu terbaik langsung lolos ke generasi berikutnya), *selection*, *crossover*, dan *mutation* hingga mencapai batas generasi.

Source Code models :

```
from __future__ import annotations
from dataclasses import dataclass
from typing import List


@dataclass
class Gene:
    inside: bool
    x: int
    y: int
    z: int
    orientation: int


@dataclass
class Individual:
    genes: List[Gene]
```

```
    fitness: float = 0.0
    loaded_value: float = 0.0
    loaded_weight: float = 0.0
    valid_count: int = 0
```

Source Code GA :

```
from __future__ import annotations
from typing import List, Tuple, Dict, Any
import random
import time
import copy

from core.models import Package, Truck
from geneticAlgorithm.models import Gene, Individual

class GeneticAlgorithm:
    def __init__(self, packages: List[Package], truck:
Truck, population_size=60,
                 generations=100, crossover_rate=0.85,
mutation_rate=0.12,
                 elite_count=2, tournament_size=3,
seed=None):
        if population_size < 2:
            raise ValueError("population_size harus >= 2")
        if generations < 1:
            raise ValueError("generations harus >= 1")
        self.packages = packages
        self.truck = truck
        self.population_size = population_size
        self.generations = generations
        self.crossover_rate = crossover_rate
        self.mutation_rate = mutation_rate
        self.elite_count = min(elite_count,
population_size - 1)
        self.tournament_size = max(2, tournament_size)
        self.rng = random.Random(seed)
        self.best_history: List[float] = []
        self.avg_history: List[float] = []
        self.population_history: List[Dict[str, float]] =
[]
        self.initial_state = None
        self.final_state = None
        self.duration = 0.0

    def _random_gene(self, i: int) -> Gene:
```

```
        p = self.packages[i]
        ori = self.rng.randrange(len(p.orientations()))
        dims = p.orientations()[ori]
        fits = dims[0] <= self.truck.w and dims[1] <=
self.truck.l and dims[2] <= self.truck.h
        inside = fits and self.rng.random() < 0.72
        if not inside:
            return Gene(False, -1, -1, -1, ori)
        return Gene(True,
                    self.rng.randint(0, self.truck.w -
dims[0]),
                    self.rng.randint(0, self.truck.l -
dims[1]),
                    self.rng.randint(0, self.truck.h -
dims[2]), ori)

    def random_individual(self) -> Individual:
        ind = Individual([self._random_gene(i) for i in
range(len(self.packages))])
        return self.repair(ind)

    def _dims(self, i: int, gene: Gene) -> Tuple[int, int,
int]:
        orientations = self.packages[i].orientations()
        return orientations[gene.orientation %
len(orientations)]

    @staticmethod
    def _overlap(a, b) -> bool:
        return (a[0] < b[0] + b[3] and a[0] + a[3] > b[0]
and
                a[1] < b[1] + b[4] and a[1] + a[4] > b[1]
and
                a[2] < b[2] + b[5] and a[2] + a[5] > b[2])

    def _box(self, i: int, g: Gene):
        w, l, h = self._dims(i, g)
        return (g.x, g.y, g.z, w, l, h)

    def _supported(self, i: int, g: Gene, placed:
List[Tuple[int, Gene]]) -> bool:
        box = self._box(i, g)
        if box[2] == 0:
            return True
        supported_cells = set()
        for j, other_gene in placed:
```

```
            other = self._box(j, other_gene)
            if other[2] + other[5] != box[2]:
                continue
            if self.packages[j].fragile:
                continue  # fragile packages cannot serve
as support
            x0 = max(box[0], other[0]); x1 = min(box[0] +
box[3], other[0] + other[3])
            y0 = max(box[1], other[1]); y1 = min(box[1] +
box[4], other[1] + other[4])
            for x in range(x0, x1):
                for y in range(y0, y1):
                    supported_cells.add((x, y))
        required = {(x, y) for x in range(box[0], box[0] +
box[3])
                    for y in range(box[1], box[1] +
box[4])}
        return required.issubset(supported_cells)

    def repair(self, individual: Individual) ->
Individual:
        ind = copy.deepcopy(individual)
        for i, g in enumerate(ind.genes):
            g.orientation %=
len(self.packages[i].orientations())
            if not g.inside:
                g.x = g.y = g.z = -1

        candidates = [i for i, g in enumerate(ind.genes)
if g.inside]
        candidates.sort(key=lambda i: (ind.genes[i].z,
-self.packages[i].value, i))
        placed: List[Tuple[int, Gene]] = []
        weight = 0.0
        accepted = []

        for i in candidates:
            p = self.packages[i]
            g = ind.genes[i]
            dims = self._dims(i, g)
            x, y, z = g.x, g.y, g.z
            w, l, h = dims
            in_bounds = (x >= 0 and y >= 0 and z >= 0 and
                         x + w <= self.truck.w and y + l
<= self.truck.l and
                         z + h <= self.truck.h)
```

```
            if not in_bounds or weight + p.weight >
self.truck.max_capacity:
                g.inside = False; g.x = g.y = g.z = -1
                continue

            box = self._box(i, g)
            if any(self._overlap(box, self._box(j, gj))
for j, gj in placed):
                g.inside = False; g.x = g.y = g.z = -1
                continue

            if z > 0 and not self._supported(i, g,
placed):
                g.inside = False; g.x = g.y = g.z = -1
                continue
            if p.fragile and any(self._box(j, gj)[2] >= z
+ h for j, gj in placed):
                g.inside = False; g.x = g.y = g.z = -1
                continue

            placed.append((i, g))
            accepted.append(i)
            weight += p.weight

        for i, gi in list(placed):
            if self.packages[i].fragile:
                bi = self._box(i, gi)
                for j, gj in list(placed):
                    if i == j:
                        continue
                    bj = self._box(j, gj)
                    if bj[2] >= bi[2] + bi[5] and
self._overlap(
                        (bi[0], bi[1], bi[2] + bi[5],
bi[3], bi[4], 1),
                        (bj[0], bj[1], bj[2], bj[3],
bj[4], bj[5])
                    ):
                        gj.inside = False; gj.x = gj.y =
gj.z = -1

        loaded = [i for i, g in enumerate(ind.genes) if
g.inside]
        ind.loaded_value = sum(self.packages[i].value for
i in loaded)
        ind.loaded_weight = sum(self.packages[i].weight
```

```
for i in loaded)
        ind.valid_count = len(loaded)
        ind.fitness = ind.loaded_value
        return ind

    def _select(self, population: List[Individual]) ->
Individual:
        contenders = self.rng.sample(population,
min(self.tournament_size, len(population)))
        return copy.deepcopy(max(contenders, key=lambda x:
x.fitness))

    def _crossover(self, a: Individual, b: Individual):
        if self.rng.random() >= self.crossover_rate:
            return copy.deepcopy(a), copy.deepcopy(b)
        c1, c2 = copy.deepcopy(a), copy.deepcopy(b)
        for i in range(len(self.packages)):
            if self.rng.random() < 0.5:
                c1.genes[i], c2.genes[i] =
copy.deepcopy(b.genes[i]), copy.deepcopy(a.genes[i])
        return self.repair(c1), self.repair(c2)

    def _mutate(self, individual: Individual) ->
Individual:
        child = copy.deepcopy(individual)
        for i, g in enumerate(child.genes):
            if self.rng.random() >= self.mutation_rate:
                continue
            p = self.packages[i]
            action = self.rng.choice(("toggle", "move",
"rotate"))
            if action == "toggle":
                g.inside = not g.inside
                if g.inside:
                    ori = g.orientation %
len(p.orientations())
                    w, l, h = p.orientations()[ori]
                    if w <= self.truck.w and l <=
self.truck.l and h <= self.truck.h:
                        g.x = self.rng.randint(0,
self.truck.w - w)
                        g.y = self.rng.randint(0,
self.truck.l - l)
                        g.z = self.rng.randint(0,
self.truck.h - h)
                    else:
```

```
                        g.inside = False
                else:
                    g.x = g.y = g.z = -1
            elif action == "rotate":
                g.orientation =
self.rng.randrange(len(p.orientations()))
                if g.inside:
                    w, l, h =
p.orientations()[g.orientation]
                    if w <= self.truck.w and l <=
self.truck.l and h <= self.truck.h:
                        g.x = min(max(g.x, 0),
self.truck.w - w)
                        g.y = min(max(g.y, 0),
self.truck.l - l)
                        g.z = min(max(g.z, 0),
self.truck.h - h)
                    else:
                        g.inside = False; g.x = g.y = g.z
= -1
            elif action == "move" and g.inside:
                w, l, h = p.orientations()[g.orientation]
                if w <= self.truck.w and l <= self.truck.l
and h <= self.truck.h:
                    g.x = self.rng.randint(0, self.truck.w
- w)
                    g.y = self.rng.randint(0, self.truck.l
- l)
                    g.z = self.rng.randint(0, self.truck.h
- h)
        return self.repair(child)

    def _snapshot(self, ind: Individual) -> Dict[str,
Any]:
        packages = {}
        for i, p in enumerate(self.packages):
            g = ind.genes[i]
            packages[p.id] = {
                "inside": g.inside,
                "coordinate": [g.x, g.y, g.z] if g.inside
else None,
                "orientation_dimensions":
list(self._dims(i, g)),
                "value": p.value, "weight": p.weight,
"fragile": p.fragile
            }
```

```
        return {"packages": packages, "objective_value":
ind.loaded_value,
                "total_weight": ind.loaded_weight,
"loaded_count": ind.valid_count}

    def run(self) -> Dict[str, Any]:
        start = time.perf_counter()
        population = [self.random_individual() for _ in
range(self.population_size)]
        self.initial_state =
self._snapshot(max(population, key=lambda x: x.fitness))
        for generation in range(self.generations + 1):
            fitnesses = [x.fitness for x in population]
            self.best_history.append(max(fitnesses))
            self.avg_history.append(sum(fitnesses) /
len(fitnesses))
            self.population_history.append({"generation":
generation,
                                            "max_fitness":
max(fitnesses),

"average_fitness": sum(fitnesses) / len(fitnesses)})
            if generation == self.generations:
                break
            population.sort(key=lambda x: x.fitness,
reverse=True)
            next_population = [copy.deepcopy(x) for x in
population[:self.elite_count]]
            while len(next_population) <
self.population_size:
                p1, p2 = self._select(population),
self._select(population)
                c1, c2 = self._crossover(p1, p2)
                next_population.append(self._mutate(c1))
                if len(next_population) <
self.population_size:

next_population.append(self._mutate(c2))
            population = next_population

        best = max(population, key=lambda x: x.fitness)
        self.final_state = self._snapshot(best)
        self.duration = time.perf_counter() - start
        return {
            "initial_state": self.initial_state,
            "final_state": self.final_state,
```

```
            "best_history": self.best_history,
            "average_history": self.avg_history,
            "population_history": self.population_history,
            "duration_seconds": self.duration,
            "population_size": self.population_size,
            "generations": self.generations,
            "final_objective": best.fitness
        }
```

2.1.4. Komponen Umum: State, Fungsi Objektif, Constraint, dan Neighbor

Hill-climbing dan Simulated Annealing memakai representasi state, fungsi objektif, constraint, dan neighbor yang sama. Genetic Algorithm memakai kelas Package dan Truck yang sama, tetapi memiliki representasi individu dan mekanisme repair sendiri. Semua komponen bersama ini berada di folder src/core.

2.1.4.1. Representasi State

Data paket disimpan dalam kelas Package dan data truk dalam kelas Truck. Posisi satu paket disimpan dalam kelas Placement yang berisi status inside, koordinat x, y, z, dan nomor orientation. Satu state adalah kelas State yang berisi daftar Placement dengan urutan yang sama dengan daftar paket.

a. Koordinat (x, y, z) adalah titik sudut kiri-bawah-belakang paket dengan berupa bilangan bulat.

b. Paket di luar truk ditandai dengan inside = False dan koordinat -1.

c. Orientasi indeks pada daftar orientations(), yaitu semua permutasi (w, l, h) yang berbeda. Paket berbentuk kubus hanya memiliki satu orientasi, sedangkan balok dengan tiga sisi berbeda memiliki enam orientasi.

d. State hanya menyimpan posisi, bukan data paket, karena data paket tidak pernah berubah. Dengan begitu proses copy() menjadi ringan, padahal proses ini dilakukan setiap kali sebuah tetangga dibangkitkan.

e. Method box(i, packages) mengubah posisi paket ke-i menjadi tuple (x, y, z, w, l, h) yang sudah memperhitungkan orientasi. Tuple ini dipakai oleh semua fungsi constraint.

f. Atribut ETA tetap disimpan pada kelas Package dan ditampilkan pada visualisasi, tetapi tidak dipakai dalam proses pencarian. Nilai sebuah susunan hanya ditentukan oleh total value paket di dalam truk, sedangkan validitasnya hanya ditentukan oleh lima constraint, yaitu paket tidak bertabrakan, tidak menembus dinding, tidak melayang, tidak menjadikan paket fragile sebagai tumpuan, dan tidak melebihi kapasitas. ETA sebenarnya bisa dipakai untuk mengatur urutan muat, misalnya paket dengan ETA lebih kecil diletakkan dekat pintu peti kemas agar dibongkar lebih dulu, tetapi aturan tersebut berada di luar persoalan ini sehingga tidak ditambahkan.

State awal dibangkitkan secara acak oleh random_state. Setiap paket mendapat orientasi acak, lalu dengan peluang 50% diletakkan pada koordinat acak di dalam truk. Paket yang ukurannya lebih besar dari truk selalu berada di luar. Karena acak, state awal umumnya masih melanggar constraint, seperti ada paket yang bertabrakan atau melayang.

Source Code

```
from __future__ import annotations
import random
from dataclasses import dataclass, field
from typing import List, Tuple


# data paket, sama dengan model GA
@dataclass(frozen=True)
class Package:
    id: str
    w: int
    l: int
    h: int
    value: float
    weight: float
    fragile: bool = False
    eta: int = 0

    def orientations(self) -> List[Tuple[int, int, int]]:
        return list(dict.fromkeys([
            (self.w, self.l, self.h), (self.w, self.h,
self.l),
            (self.l, self.w, self.h), (self.l, self.h,
```

```
self.w),
            (self.h, self.w, self.l), (self.h, self.l,
self.w)
        ]))


# data truk, sama dengan model GA
@dataclass(frozen=True)
class Truck:
    w: int
    l: int
    h: int
    max_capacity: float


# posisi dan orientasi satu paket
@dataclass
class Placement:
    inside: bool
    x: int = -1
    y: int = -1
    z: int = -1
    orientation: int = 0


# satu state = placement untuk tiap paket
@dataclass
class State:
    placements: List[Placement] =
field(default_factory=list)

    def copy(self) -> State:
        return State([Placement(p.inside, p.x, p.y, p.z,
p.orientation)
            for p in self.placements])

    def dims(self, i: int, packages: List[Package]) ->
Tuple[int, int, int]:
        ori = packages[i].orientations()
        return ori[self.placements[i].orientation %
len(ori)]

    def box(self, i: int, packages: List[Package]) ->
Tuple[int, int, int, int, int, int]:
        p = self.placements[i]
        w, l, h = self.dims(i, packages)
```

```
        return (p.x, p.y, p.z, w, l, h)

    def inside_indices(self) -> List[int]:
        return [i for i, p in enumerate(self.placements)
if p.inside]


# state awal acak
def random_state(packages: List[Package], truck: Truck,
rng: random.Random) -> State:
    placements = []
    for pkg in packages:
        ori = rng.randrange(len(pkg.orientations()))
        w, l, h = pkg.orientations()[ori]
        fits = w <= truck.w and l <= truck.l and h <=
truck.h
        if fits and rng.random() < 0.5:
            placements.append(Placement(True,
rng.randint(0, truck.w - w),
                rng.randint(0, truck.l - l),
rng.randint(0, truck.h - h), ori))
        else:
            placements.append(Placement(False,
orientation=ori))
    return State(placements)
```

2.1.4.2. Fungsi Objektif

Fungsi objektif dimaksimalkan dan didefinisikan sebagai total value paket di dalam truk dikurangi penalti pelanggaran constraint.

$$f(s) = \sum_{i \in inside} value_i - M \times (V_{overlap} + V_{dinding} + N_{melayang} + N_{fragile} + W_{lebih})$$

Nilai M adalah total value seluruh paket ditambah satu. Dengan nilai ini, satu unit pelanggaran saja sudah lebih besar daripada value terbaik yang mungkin dicapai. Akibatnya state yang valid selalu lebih baik daripada state yang melanggar, dan objektif yang bernilai positif pasti berasal dari state tanpa pelanggaran. Di antara state yang sama-sama melanggar, state dengan pelanggaran lebih sedikit tetap mendapat nilai lebih tinggi, sehingga algoritma masih bisa bergerak menuju state yang valid.

Semua constraint dihitung sebagai besaran, bukan hanya benar atau salah. Misalnya overlap dihitung sebagai volume tabrakan. Jika volume tabrakan turun dari 8 menjadi 2, nilai objektif ikut naik walaupun state belum valid. Hal ini membuat algoritma dapat melihat kemajuan kecil.

Semua constraint diberi bobot yang sama karena semuanya wajib dipenuhi dan tidak ada yang lebih penting dari yang lain.

2.1.4.3. Constraint

Semua fungsi constraint menerima paket dalam bentuk tuple (x, y, z, w, l, h) dan hanya memeriksa paket yang berada di dalam truk.

Constraint	Fungsi	Cara menghitung	Satuan
Paket bertabrakan	total_overlap	Jumlah volume irisan setiap pasangan paket. Irisan dihitung per sumbu, lalu ketiga panjang irisan dikalikan.	volume
Menembus dinding	total_out_of_bounds	Volume paket dikurangi volume irisan paket dengan ruang truk	volume
Paket melayang	floating_count	Jumlah paket yang tidak berada di lantai dan tidak memiliki paket lain yang atapnya tepat di bawah alasnya dengan luas sentuhan lebih dari nol	paket
Fragile menjadi tumpuan	fragile_support_count	Jumlah paket yang alasnya menyentuh atap paket fragile	paket
Kelebihan berat	overweight	Total berat dikurangi kapasitas, minimal nol	berat

Definisi melayang mengikuti spesifikasi. Paket cukup memiliki satu titik tumpuan pada alasnya agar tidak dianggap melayang, sesuai dengan jawaban asisten pada sesi QnA. Paket yang hanya bersentuhan pada sisi samping tidak dihitung sebagai tumpuan karena luas sentuhannya nol.

Pada fragile_support_count, yang dihitung adalah jumlah paket yang menumpang, bukan jumlah paket fragile yang tertimpa. Jika satu paket fragile ditimpa dua paket, nilainya 2. Ketika salah satu paket dipindahkan, nilainya turun menjadi 1 sehingga perbaikan tersebut terlihat pada fungsi objektif.

Source Code

```
from itertools import combinations

from core.models import Package, State, Truck

# box = (x, y, z, w, l, h)


# panjang irisan di satu sumbu
def _axis_overlap(a0, a_len, b0, b_len):
    return max(0, min(a0 + a_len, b0 + b_len) - max(a0,
b0))


# volume irisan dua box
def overlap_volume(a, b):
    dx = _axis_overlap(a[0], a[3], b[0], b[3])
    dy = _axis_overlap(a[1], a[4], b[1], b[4])
    dz = _axis_overlap(a[2], a[5], b[2], b[5])
    return dx * dy * dz


# total overlap semua pasangan paket
def total_overlap(boxes):
    return sum(overlap_volume(a, b) for a, b in
combinations(boxes, 2))


# luas irisan alas a dengan atap b
def _contact_area(a, b):
    dx = _axis_overlap(a[0], a[3], b[0], b[3])
    dy = _axis_overlap(a[1], a[4], b[1], b[4])
    return dx * dy


# paket di lantai atau ditopang paket lain
def is_supported(box, boxes):
    if box[2] == 0:
        return True
```

```
    for other in boxes:
        if other[2] + other[5] == box[2] and
_contact_area(box, other) > 0:
            return True
    return False


# jumlah paket melayang
def floating_count(boxes):
    return sum(1 for b in boxes if not is_supported(b,
boxes))


# jumlah paket yang menumpang di atas paket fragile
def fragile_support_count(boxes, fragile):
    count = 0
    for i, below in enumerate(boxes):
        if not fragile[i]:
            continue
        for j, above in enumerate(boxes):
            if i != j and below[2] + below[5] ==
above[2] and _contact_area(above, below) > 0:
                count += 1
    return count


# volume paket yang keluar dari truk
def out_of_bounds_volume(box, truck_dims):
    inside = overlap_volume(box, (0, 0, 0) +
tuple(truck_dims))
    return box[3] * box[4] * box[5] - inside


# total volume tembus dinding
def total_out_of_bounds(boxes, truck_dims):
    return sum(out_of_bounds_volume(b, truck_dims) for
b in boxes)


# kelebihan berat di atas kapasitas
def overweight(total_weight, max_capacity):
    return max(0, total_weight - max_capacity)


# bobot penalti, lebih besar dari total value
def penalty_weight(packages):
```

```
    return sum(p.value for p in packages) + 1


# rincian pelanggaran tiap constraint
def violations(state: State, packages: list[Package],
truck: Truck):
    idx = state.inside_indices()
    boxes = [state.box(i, packages) for i in idx]
    fragile = [packages[i].fragile for i in idx]
    weight = sum(packages[i].weight for i in idx)
    return {
        "overlap": total_overlap(boxes),
        "out_of_bounds": total_out_of_bounds(boxes,
(truck.w, truck.l, truck.h)),
        "floating": floating_count(boxes),
        "fragile": fragile_support_count(boxes,
fragile),
        "overweight": overweight(weight,
truck.max_capacity),
    }


# nilai objektif, makin besar makin baik
def objective(state: State, packages: list[Package],
truck: Truck):
    value = sum(packages[i].value for i in
state.inside_indices())
    total = sum(violations(state, packages,
truck).values())
    return value - penalty_weight(packages) * total
```

2.1.4.4. Neighbor

Terdapat tiga jenis langkah neighbor. Setiap langkah membuat salinan state baru sehingga state asal tidak berubah.

a. swap(state, i, j) menukar status inside dan koordinat dua paket. Orientasi tetap mengikuti paket masing-masing. Jika satu paket berada di dalam dan yang lain di luar, langkah ini sekaligus mengganti paket yang dimuat.

b. move(state, i, pos) memindahkan paket ke koordinat pos. Jika pos bernilai None, paket dikeluarkan dari truk.

c. rotate(state, i, axis, packages) merotasi paket 90 derajat terhadap sumbu x, y, atau z. Rotasi terhadap satu sumbu sama dengan menukar dua dimensi lainnya. Rotasi terhadap sumbu z menukar w dan l, terhadap sumbu x menukar l dan h, dan terhadap sumbu y menukar w dan h.

Fungsi all_neighbors membangkitkan semua tetangga dan dipakai oleh steepest ascent dan sideways move. Fungsi random_neighbor membangkitkan satu tetangga acak dan dipakai stochastic hill-climbing dan Simulated Annealing. Swap antara dua paket yang sama-sama di luar truk dan rotasi yang tidak mengubah orientasi tidak dibangkitkan karena tidak mengubah state.

2.1.4.5. Heuristik Kandidat Posisi

Langkah move dapat memindahkan paket ke koordinat mana pun. Truk berukuran 10 x 20 x 5 memiliki 1000 titik, sehingga mencoba semua titik untuk setiap paket akan membuat jumlah tetangga sangat besar. Oleh karena itu, kami memakai heuristik sendiri pada fungsi candidate_positions yang hanya mencoba posisi berikut.

a. Pojok truk (0, 0, 0).

b. Posisi tepat di sisi +x, sisi +y, dan atas setiap paket lain di dalam truk, yaitu (x + w, y, z), (x, y + l, z), dan (x, y, z + h).

c. Posisi hasil geser satu satuan ke enam arah dari posisi paket saat ini.

Posisi yang membuat paket keluar dari truk langsung dibuang. Arah positif dipilih karena koordinat paket adalah titik sudut terkecilnya, sehingga posisi yang menempel di sisi positif paket lain dapat dihitung langsung dari ukuran paket tersebut tanpa bergantung pada ukuran paket yang dipindahkan. Penyusunan dimulai dari pojok (0, 0, 0) dan melebar ke arah positif. Posisi di atas paket lain menjamin paket yang dipindahkan memiliki tumpuan. Posisi arah lain tetap dapat dicapai melalui pergeseran satuan.

Source Code

```
import random
from typing import Iterator, List, Tuple
```

```
from core.models import Package, State, Truck

# sumbu rotasi -> pasangan dimensi yang ditukar
AXIS_SWAP = {"x": (1, 2), "y": (0, 2), "z": (0, 1)}


# tukar posisi dua paket
def swap(state: State, i: int, j: int) -> State:
    new = state.copy()
    a, b = new.placements[i], new.placements[j]
    a.inside, b.inside = b.inside, a.inside
    a.x, b.x = b.x, a.x
    a.y, b.y = b.y, a.y
    a.z, b.z = b.z, a.z
    return new


# pindahkan paket ke koordinat lain
def move(state: State, i: int, pos) -> State:
    new = state.copy()
    p = new.placements[i]
    if pos is None:
        p.inside, p.x, p.y, p.z = False, -1, -1, -1
    else:
        p.inside, (p.x, p.y, p.z) = True, pos
    return new


# rotasi 90 derajat terhadap satu sumbu
def rotate(state: State, i: int, axis: str, packages:
List[Package]) -> State:
    dims = list(state.dims(i, packages))
    a, b = AXIS_SWAP[axis]
    dims[a], dims[b] = dims[b], dims[a]
    new = state.copy()
    new.placements[i].orientation =
packages[i].orientations().index(tuple(dims))
    return new


# kandidat posisi: pojok truk dan sisi paket lain
def candidate_positions(state: State, i: int, packages:
List[Package],
    truck: Truck) -> List[Tuple[int, int, int]]:
    w, l, h = state.dims(i, packages)
```

```
    spots = {(0, 0, 0)}
    for j in state.inside_indices():
        if j == i:
            continue
        x, y, z, bw, bl, bh = state.box(j, packages)
        spots.update({(x + bw, y, z), (x, y + bl, z),
(x, y, z + bh)})
    p = state.placements[i]
    if p.inside:
        for dx, dy, dz in ((1, 0, 0), (-1, 0, 0), (0,
1, 0), (0, -1, 0), (0, 0, 1), (0, 0, -1)):
            spots.add((p.x + dx, p.y + dy, p.z + dz))
        spots.discard((p.x, p.y, p.z))
    return sorted(s for s in spots
        if 0 <= s[0] <= truck.w - w and 0 <= s[1] <=
truck.l - l
        and 0 <= s[2] <= truck.h - h)


# semua tetangga dari satu state
def all_neighbors(state: State, packages:
List[Package], truck: Truck) -> Iterator[State]:
    n = len(packages)
    for i in range(n):
        for j in range(i + 1, n):
            if state.placements[i].inside or
state.placements[j].inside:
                yield swap(state, i, j)
    for i in range(n):
        for pos in candidate_positions(state, i,
packages, truck):
            yield move(state, i, pos)
        if state.placements[i].inside:
            yield move(state, i, None)
    for i in range(n):
        for axis in AXIS_SWAP:
            new = rotate(state, i, axis, packages)
            if new.placements[i].orientation !=
state.placements[i].orientation:
                yield new


# satu tetangga acak
def random_neighbor(state: State, packages:
List[Package], truck: Truck,
        rng: random.Random) -> State:
```

```
    n = len(packages)
    action = rng.choice(("swap", "move", "rotate"))
    if action == "swap" and n > 1:
        i, j = rng.sample(range(n), 2)
        return swap(state, i, j)
    i = rng.randrange(n)
    if action == "rotate":
        return rotate(state, i,
rng.choice(tuple(AXIS_SWAP)), packages)
    spots = candidate_positions(state, i, packages,
truck)
    if state.placements[i].inside and (not spots or
rng.random() < 0.1):
        return move(state, i, None)
    return move(state, i, rng.choice(spots)) if spots
else state.copy()
```

2.2. Hasil Eksperimen

Semua eksperimen dijalankan dengan python run_all.py menggunakan data input/contoh.json. Data ini berisi 15 paket yang 2 di antaranya *fragile* dengan total value 533 dan total berat 72, serta truk berukuran 6 x 8 x 4 dengan kapasitas 45. Karena total berat melebihi batas, tidak semua paket dapat dimuat, sehingga nilai 533 tidak mungkin dicapai. Setiap run memakai seed yang berbeda. Seluruh state awal, state akhir, visualisasi 3D, dan plot setiap run tersimpan di folder output.

Nilai M pada data ini adalah 534. Semua nilai objektif akhir di bawah akan bernilai positif, sehingga seluruh state akhir tidak melanggar constraint.

2.2.1. Algoritma Hill-Climbing

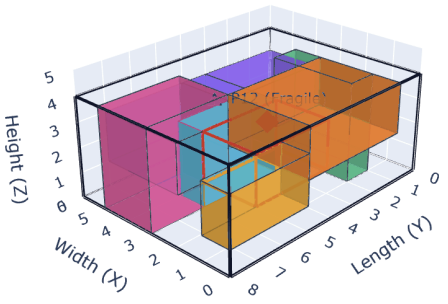
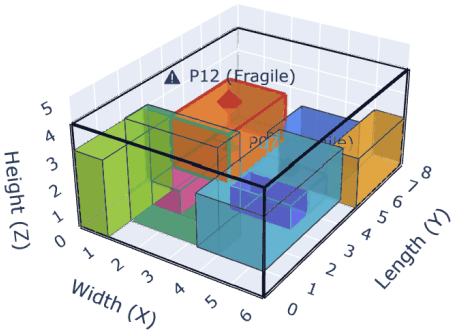
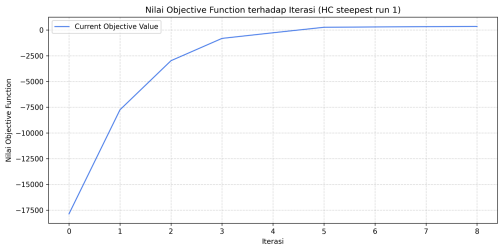
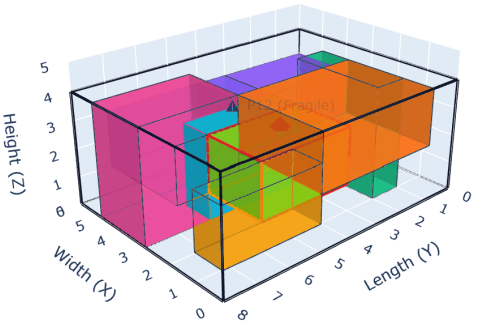
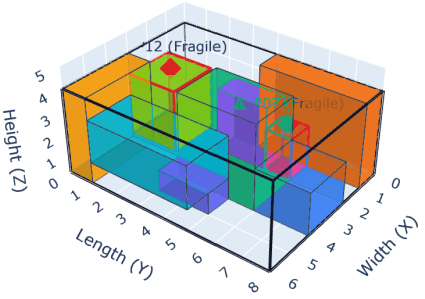
Varian	Parameter	Run	Objektif akhir	Iterasi	Durasi(s)	Keterangan
Steepest ascent	-	1	351	8	0.26	
	-	2	375	6	0.18	
	-	3	363	10	0.32	
Sideways move	max sideways 100	1	399	112	4.85	100 sideways move

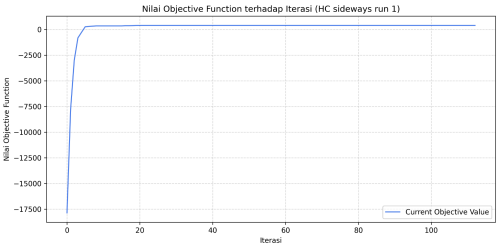
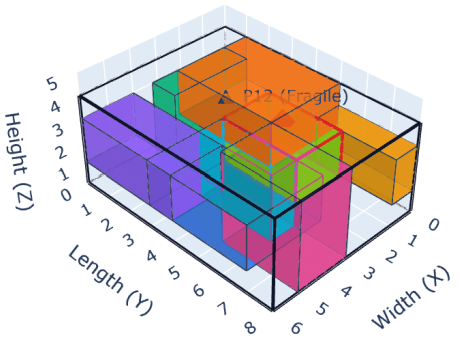
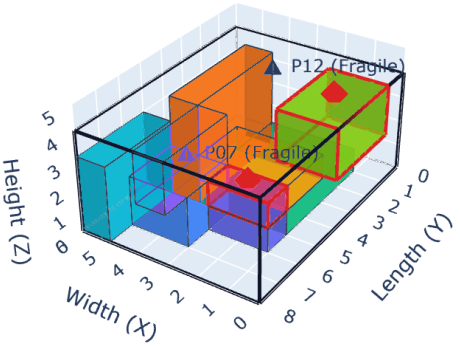
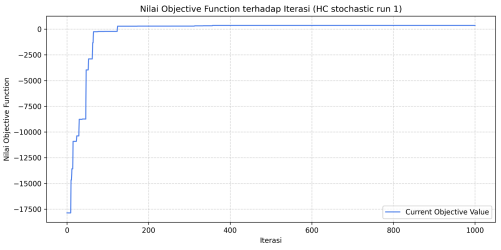
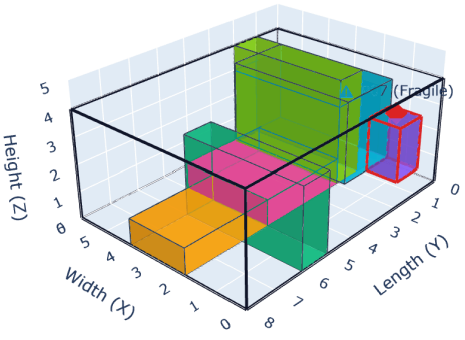
	max sideways 100	2	410	109	5.58	100 sideways move
	max sideways 100	3	363	110	5.69	100 sideways move
Stochastic	1000 iterasi	1	361	1000	0.16	
	1000 iterasi	2	351	1000	0.15	
	1000 iterasi	3	318	1000	0.15	
Random restart	max restart 10	1	413	80	3.33	10 restart
	max restart 10	2	412	81	3.42	10 restart
	max restart 10	3	410	74	2.90	10 restart

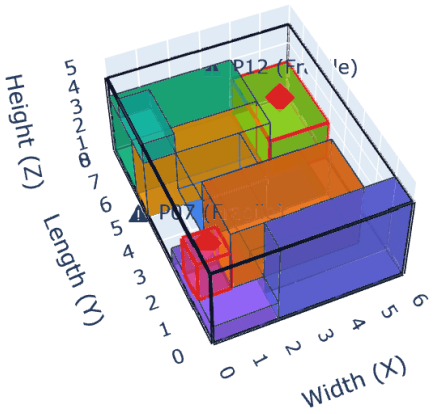
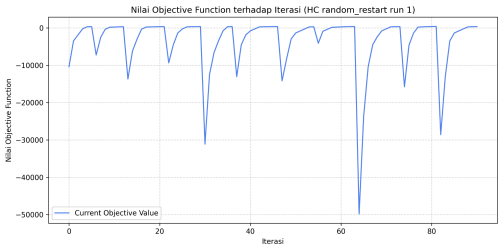
Banyak iterasi per restart pada random restart adalah sebagai berikut.

Run	**Iterasi per restart**
1	5, 6, 8, 7, 6, 9, 7, 8, 9, 7, 8
2	9, 8, 9, 7, 8, 8, 7, 7, 6, 5, 7
3	7, 7, 5, 6, 8, 9, 7, 5, 8, 6, 6

Keterangan	File/path	Gambar

State awal steepest ascent run 1	output/hc/hc_steepest_run1_awal.html	
State akhir steepest ascent run 1	output/hc/hc_steepest_run1_akhir.html	
Plot nilai objektif terhadap iterasi steepest ascent run 1	output/hc/hc_steepest_run1_objective.png	
State awal sideways move run 1	output\hc\hc_sideways_run1_awal.html	
State akhir sideways move run 1	output\hc\hc_sideways_run1_akhir.html	

Plot nilai objektif terhadap iterasi sideways move run 1	output\hc\hc_sideways_run1_objective.png	
State awal stochastic run 1	output\hc\hc_stochastic_run1_awal.html	
State akhir stochastic run 1	output\hc\hc_stochastic_run1_akhir.html	
Plot nilai objektif terhadap iterasi stochastic run 1	output\hc\hc_stochastic_run1_objective.png	
State awal random restart run 1	output\hc\hc_random_restart_run1_awal.html	

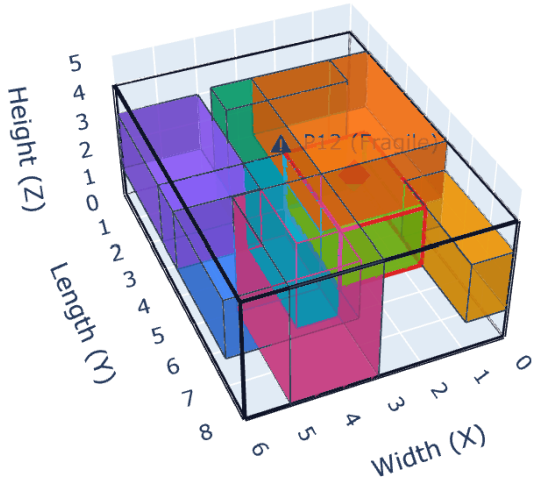
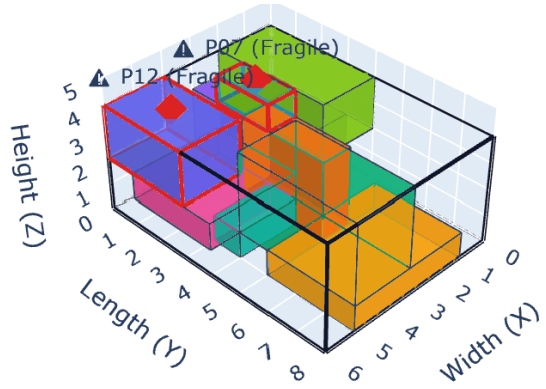
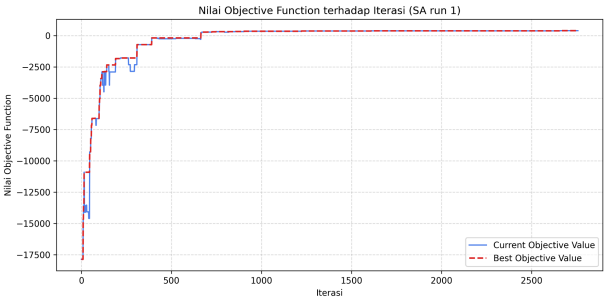
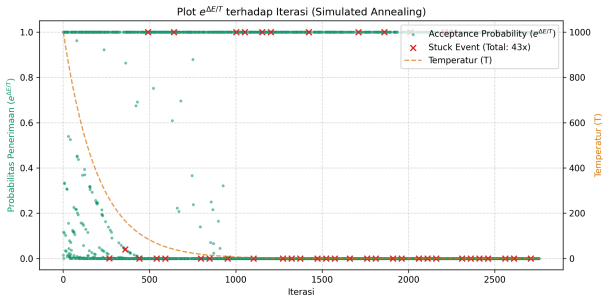
State akhir random restart run 1	output\hc\hc_random_restart_run1_akhir.html	
Plot nilai objektif terhadap iterasi random restart run 1	output\hc\hc_random_restart_run1_objective.png	

2.2.2. Algoritma Simulated Annealing

Simulated Annealing dijalankan dengan parameter default, yaitu suhu awal 1000, cooling rate 0.995, suhu minimum 0.001, dan jadwal pendinginan geometric. Pencarian berhenti setelah 2757 iterasi karena suhu sudah turun di bawah suhu minimum. Objektif akhir yang dicatat adalah nilai state terbaik selama pencarian.

Run	Objektif akhir	Iterasi	Durasi(s)	Frekuensi stuck
1	398	2757	0.50	43
2	408	2757	0.47	47
3	368	2757	0.51	48

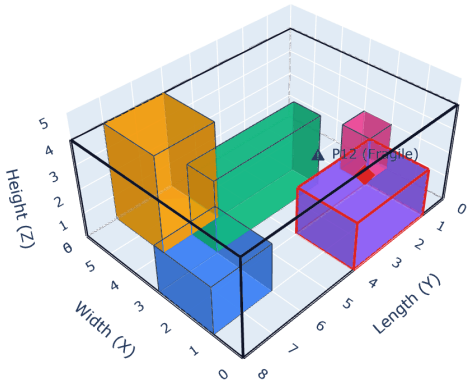
Keterangan	File/path	Gambar

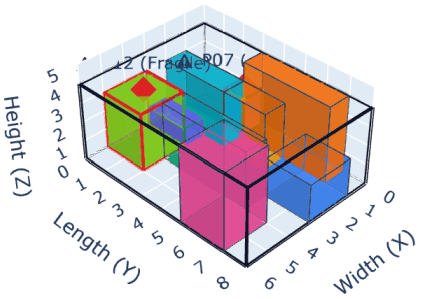
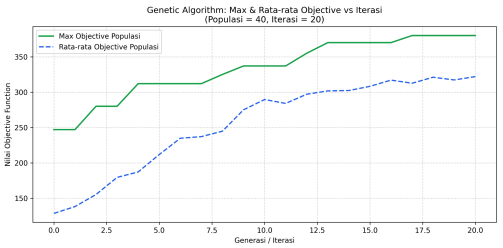
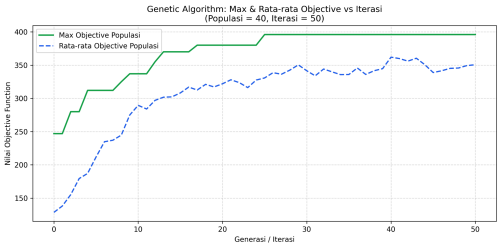
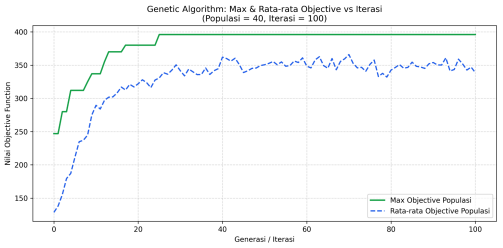
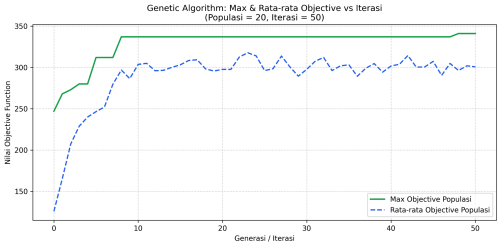
State awal Simulated Annealing run 1	output/sa/sa_run1_awal.html	
State akhir Simulated Annealing run 1	output/sa/sa_run1_akhir.html	
Plot nilai objektif terhadap iterasi Simulated Annealing run 1	output/sa/sa_run1_objective.png	
Plot e^(ΔE/T) terhadap iterasi Simulated Annealing run 1	output/sa/sa_run1_acceptance.png	

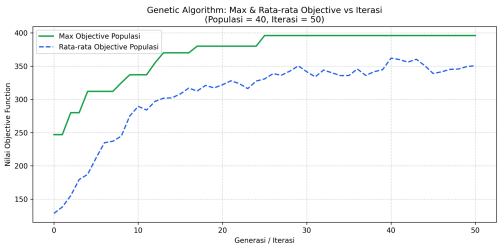
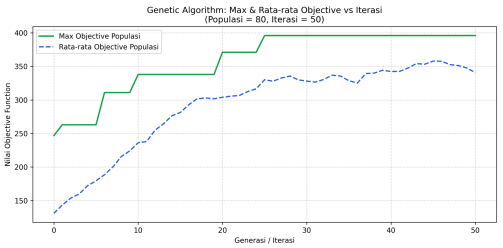
2.2.3. Genetic Algorithm

Pada eksperimen pertama, jumlah populasi dijadikan kontrol sebesar 40 dengan variasi banyak iterasi 20, 50, dan 100. Pada eksperimen kedua, banyak iterasi dijadikan kontrol sebesar 50 dengan variasi jumlah populasi 20, 40, dan 80. Setiap konfigurasi dijalankan 3 kali.

Kontrol	**Populasi**	**Iterasi**	**Objektif run 1**	**Objektif run 2**	**Objektif run 3**	**Durasi rata-rata (s)**
Populasi	40	20	380	360	388	0.81
		50	396	388	413	2.13
		100	396	413	413	4.44
Iterasi	20	50	341	399	399	1.04
	40		396	388	413	2.22
	80		396	374	399	4.52

Keterangan	File/path	Gambar
State awal Genetic Algorithm populasi 40, iterasi 100, run 1	output/ga/ga_kontrol_populasi_pop40_gen100_run1_awal.html	

State akhir Genetic Algorithm populasi 40, iterasi 100, run 1	output/ga/ga_kontrol_populasi_pop40_gen100_run1_akhir.html	
Populasi 40, iterasi 20, run 1	output/ga/ga_kontrol_populasi_pop40_gen20_run1_fitness.png	
Populasi 40, iterasi 50, run 1	output/ga/ga_kontrol_populasi_pop40_gen50_run1_fitness.png	
Populasi 40, iterasi 100, run 1	output/ga/ga_kontrol_populasi_pop40_gen100_run1_fitness.png	
Populasi 20, iterasi 50, run 1	output/ga/ga_kontrol_iterasi_pop20_gen50_run1_fitness.png	

Populasi 40, iterasi 50, run 1	output/ga/ga_kontrol_iterasi_pop40_gen50_run1_fitness.png	
Populasi 80, iterasi 50, run 1	output/ga/ga_kontrol_iterasi_pop80_gen50_run1_fitness.png	

2.3. Analisis Eksperimen

Ringkasan hasil setiap algoritma ditunjukkan pada tabel berikut. Untuk Genetic Algorithm, yang dimasukkan adalah konfigurasi populasi 40 dan iterasi 100 karena konfigurasi tersebut memberikan hasil terbaik.

Algoritma	Rata-rata objektif	Minimum	Maksimum	Rata-rata durasi (s)
Random restart	411.7	410	413	3.22
Genetic Algorithm (populasi 40, iterasi 100)	407.3	396	413	4.44
Simulated Annealing	391.3	368	408	0.49
Sideways move	390.7	363	410	5.37
Steepest ascent	363.0	351	375	0.25
Stochastic	343.3	318	361	0.15

2.3.1. Kedekatan dengan Global Optimum

Nilai terbaik yang ditemukan dari seluruh eksperimen adalah 413, dicapai oleh random restart dan Genetic Algorithm. Global optimum tidak dapat dipastikan karena ruang state terlalu besar untuk dicari secara menyeluruh, tetapi nilai 413 muncul berulang kali dari algoritma dan seed yang berbeda sehingga kemungkinan besar nilai ini sudah dekat dengan global optimum.

Random restart paling dekat dengan nilai tersebut karena algoritma mencoba 11 titik awal yang berbeda dan mengambil yang terbaik. Steepest ascent dan stochastic paling jauh karena keduanya hanya bergerak naik dan berhenti pada local optimum pertama yang ditemui. Pada steepest ascent, pencarian berhenti setelah 6 sampai 10 iterasi, yang menunjukkan bahwa local optimum pada persoalan ini cukup banyak dan mudah dicapai.

2.3.2. Perbandingan Hasil Antar Algoritma

Varian hill-climbing yang dapat keluar dari local optimum memberikan hasil lebih baik daripada steepest ascent biasa. Sideways move menaikkan rata-rata dari 363 menjadi 390.7, karena banyak susunan yang memiliki value sama, misalnya paket yang digeser ke tempat kosong. Pada ketiga run, batas 100 sideways move selalu habis, sehingga algoritma lebih sering berjalan di daerah datar daripada naik.

Simulated Annealing memberikan hasil yang setara dengan sideways move. Pada suhu tinggi, algoritma menerima banyak langkah yang memperburuk state sehingga dapat keluar dari local optimum, lalu bersikap seperti stochastic hill-climbing ketika suhu sudah rendah. Hasilnya jauh lebih baik daripada stochastic hill-climbing biasa walaupun keduanya sama-sama hanya membangkitkan satu tetangga acak per iterasi.

Genetic Algorithm memberikan hasil yang hampir sama dengan random restart. Pendekatan repair membuat setiap individu selalu valid, sehingga seluruh pencarian dipakai untuk menaikkan value. Kekurangannya, aturan tumpuan pada repair lebih ketat daripada spesifikasi, sehingga beberapa susunan yang sebenarnya valid tidak dapat dihasilkan.

2.3.3. Perbandingan Durasi

Stochastic hill-climbing dan steepest ascent adalah yang tercepat, yaitu di bawah 0.4 detik. Satu iterasi steepest ascent mengevaluasi ratusan tetangga, tetapi jumlah iterasinya sedikit. Sebaliknya, satu iterasi stochastic hanya mengevaluasi satu tetangga.

Sideways move adalah varian hill-climbing yang paling lambat, sekitar 20 kali lebih lama daripada steepest ascent, karena setiap sideways move tetap mengevaluasi semua tetangga. Random restart sekitar 13 kali lebih lama daripada steepest ascent karena memang menjalankan steepest ascent sampai 11 kali. Simulated Annealing cukup cepat (sekitar 0.5 detik) walaupun menjalankan 2757 iterasi, karena setiap iterasi hanya

mengevaluasi satu tetangga. Durasi Genetic Algorithm naik hampir sebanding dengan jumlah populasi dan banyak iterasi.

2.3.4. Konsistensi Hasil

Random restart adalah algoritma yang paling konsisten dengan selisih hasil hanya 3 (410 sampai 413). Algoritma lain memiliki selisih yang jauh lebih besar, yaitu 24 untuk steepest ascent, 40 untuk Simulated Annealing, 43 untuk stochastic, dan 47 untuk sideways move. Algoritma yang hanya bergantung pada satu state awal sangat dipengaruhi oleh state awal tersebut, sedangkan random restart mengurangi pengaruh itu dengan mencoba banyak state awal.

2.3.5. Pengaruh Iterasi dan Populasi pada Genetic Algorithm

Dengan populasi tetap 40, menambah iterasi meningkatkan hasil. Rata-rata objektif naik dari 376 pada 20 iterasi menjadi 399 pada 50 iterasi dan 407.3 pada 100 iterasi. Iterasi yang lebih banyak memberi kesempatan lebih banyak bagi crossover dan mutasi untuk menemukan susunan yang lebih baik, sementara elitism menjaga individu terbaik agar tidak hilang.

Dengan iterasi tetap 50, rata-rata objektif adalah 379.7 untuk populasi 20, 399 untuk populasi 40, dan 389.7 untuk populasi 80. Populasi 20 menghasilkan rata-rata terendah karena keragamannya kecil. Populasi 80 tidak lebih baik daripada populasi 40 pada percobaan ini. Dengan hanya 3 run per konfigurasi, perbedaan ini kemungkinan besar dipengaruhi oleh seed, sehingga dapat disimpulkan bahwa menambah populasi di atas 40 tidak memberikan peningkatan yang jelas tetapi menambah durasi hampir dua kali lipat.

III. Kesimpulan dan Saran

3.1. Kesimpulan

1) Persoalan penyusunan paket pada peti kemas truk dapat diselesaikan dengan algoritma local search. Seluruh algoritma yang diimplementasikan menghasilkan state akhir yang memenuhi semua constraint.

2) Fungsi objektif berbasis penalti dengan M sebesar total value seluruh paket ditambah satu membuat state valid selalu lebih baik daripada state yang melanggar, tetapi tetap memungkinkan algoritma bergerak dari state awal acak yang belum valid.

3) Random restart hill climbing memberikan hasil terbaik dan paling konsisten pada eksperimen ini, dengan rata-rata objektif 411.7. Genetic Algorithm dengan populasi 40 dan 100 iterasi memberikan hasil yang hampir serupa.

4) Steepest ascent dan stochastic hill-climbing paling cepat, tetapi mudah berhenti pada local optimum. Sideways move dan Simulated Annealing dapat keluar dari sebagian local optimum dengan hasil rata-rata sekitar 391.

5) Pada Genetic Algorithm, menambah banyak iterasi meningkatkan hasil, sedangkan menambah populasi di atas 40 tidak memberikan peningkatan yang jelas dan hanya menambah durasi.

3.2. Saran

1) Waktu evaluasi steepest ascent naik cepat ketika jumlah paket bertambah karena jumlah tetangga dan perhitungan overlap sama-sama bertambah secara kuadratik. Untuk input yang lebih besar, jumlah tetangga yang dievaluasi pada setiap iterasi sebaiknya dibatasi, misalnya dengan mengambil sampel acak dari seluruh tetangga.

2) Aturan tumpuan pada repair Genetic Algorithm sebaiknya disamakan dengan definisi melayang pada spesifikasi agar ruang solusi ketiga algoritma sama.

3) Eksperimen sebaiknya dijalankan dengan lebih banyak run dan beberapa data input dengan ukuran berbeda agar perbandingan antar algoritma tidak terlalu dipengaruhi oleh seed.

IV. Pembagian Tugas

No	NIM	Nama Lengkap	Mengerjakan Bagian
1	13524120	Jonathan Alveraldo Bangun	Simulated Annealing, Plot, Visualization 3D, dan Laporan
2	13524126	Ramadhian Nabil Firdaus Gumay	Fungsi Objektif dan Constraint, Representasi State, Neighbor, Seluruh Varian Hill-Climbing, Integrasi, Eksperimen, dan Laporan
3	13524144	Jonathan Harijadi	Genetic Algorithm, dan Laporan

V. Referensi

[1] Asisten Lab I. A., “Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027”. [Accessed: Okt. 20, 2026].

[2] S. Russell dan P. Norvig, Artificial Intelligence: A Modern Approach, edisi ke-3, Pearson, 2010.

VI. Lampiran Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:
Nama/NIM : Jonathan Alveraldo Bangun / 13524120,
Ramadhian Nabil Firdaus Gumay / 13524126,
Jonathan Harijadi / 13524144
Fakultas/Sekolah : STEI-K
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1 IF3170 Inteligensi Artifisial

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak*	1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	1

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Jonathan Alveraldo Bangun
NIM 13524120



Ramadhian Nabil Firdaus Gumay

NIM 13524126



Jonathan Harijadi

NIM 13524144